

系统开发工具基础

第 1 周实验报告

姓名：朱元清 学号：25020007193

2026 年 8 月 24 日

本报告记录第 1 周四道实验题的执行命令与验证结果，内容覆盖 Shell 文件整理、管道文本统计、Git 合并冲突处理和 LaTeX 文档构建。

表 1 第 1 周实验完成情况

实验	主题	关键工具	状态
1	含空格文件名的批量整理	Shell、find、chmod	✓ 完成
2	随机访问日志统计	awk、sort、head	✓ 完成
3	制造并解决一次合并冲突	Git、merge	✓ 完成
4	修复并构建一页技术说明	LaTeX、交叉引用	✓ 完成

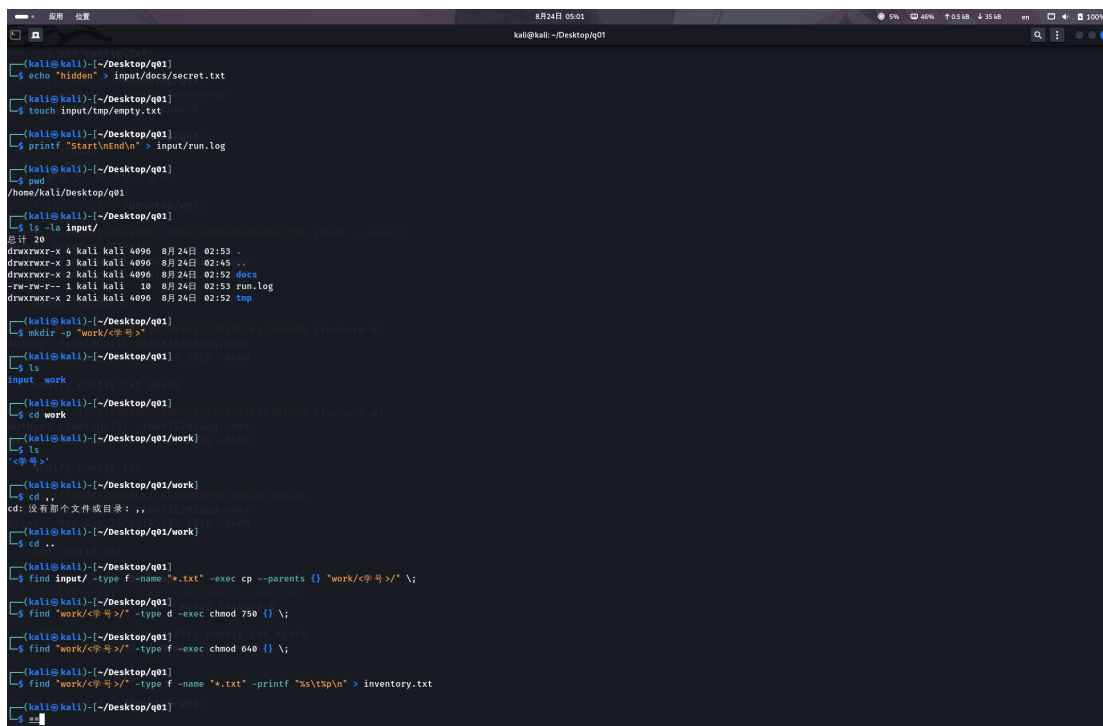
实验一 含空格文件名的批量整理 (Shell 基础与文件系统)

执行命令

```
mkdir q01 && cd q01
mkdir -p input/{docs,tmp}
printf 'alpha\nbeta\n' > "input/docs/notes one.txt"
printf 'hidden\n' > input/docs/.secret.txt
: > input/tmp/empty.txt
printf 'start\nend\n' > input/run.log

pwd
ls -la input

STUDENT_ID=25020007193
mkdir -p "work/$STUDENT_ID"
find input -type f -name '*.txt' \
    -exec cp --parents {} "work/$STUDENT_ID/" \;
find "work/$STUDENT_ID" -type d -exec chmod 750 {} +
find "work/$STUDENT_ID" -type f -exec chmod 640 {} +
find "work/$STUDENT_ID/input" -type f -name '*.txt' \
    -printf '%P\t%s\n' | sort > inventory.txt
cat inventory.txt
```



```
kali@kali: ~/Desktop/q01
$ echo "hidden" > input/docs/secret.txt
$ touch input/tmp/empty.txt
$ printf "start\nend\n" > input/run.log
$ pwd
/home/kali/Desktop/q01
$ ls -la input/
total 20
drwxrwxr-x 4 kali kali 4096 8月24日 02:53
drwxrwxr-x 3 kali kali 4096 8月24日 02:45 docs
drwxrwxr-x 2 kali kali 4096 8月24日 02:52 tmp
-rw-rw-r-- 1 kali kali 10 8月24日 02:53 run.log
drwxrwxr-x 2 kali kali 4096 8月24日 02:52 tmp
$ mkdir -p "work/<学号>"
$ ls
input work
$ cd work
$ ls
<学号>
$ cd ..
cd: 没有那个文件或目录: ..
$ cd ..
$ find input/ -type f -name "*.txt" -exec cp --parents {} "work/<学号>/" \;
$ find "work/<学号>/" -type d -exec chmod 750 {} \;
$ find "work/<学号>/" -type f -exec chmod 640 {} \;
$ find "work/<学号>/" -type f -name "*.txt" -printf "%P\t%s\n" > inventory.txt
$ cat inventory.txt
```

图 1 实验一运行结果

结果说明

完成目录和测试文件创建，复制后的目录结构得到保留；目录权限为 750，普通文件权限为 640，inventory.txt 列出了各文本文件的相对路径和字节数。

实验二 随机访问日志统计 (Shell 管道与文本处理)

执行命令

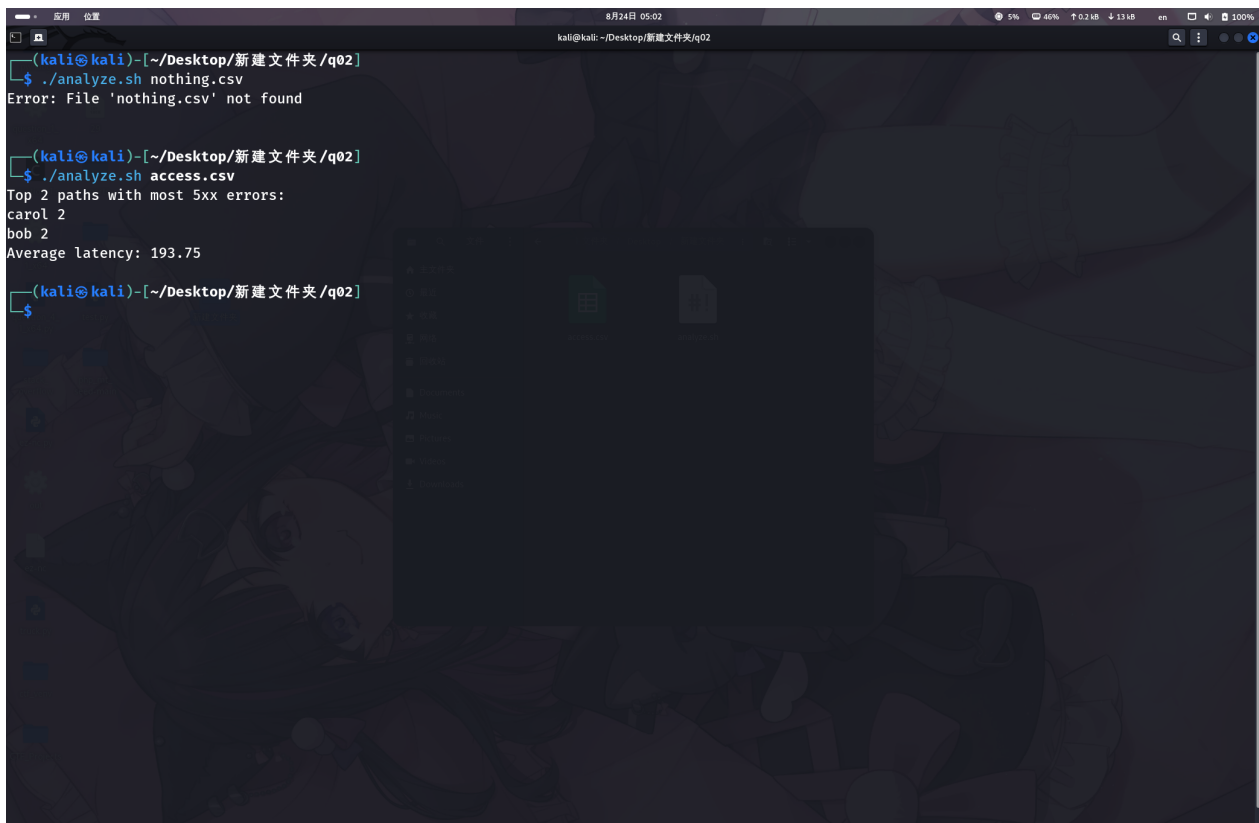
```
mkdir q02 && cd q02
cat > access.csv <<'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF

cat > analyze.sh <<'EOF'
#!/bin/bash
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <csv_file>" >&2
    exit 1
fi
csv_file="$1"
if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' not found" >&2
    exit 1
fi

echo "Top 2 paths with most 5xx errors:"
awk -F, 'NR>1 && $4>=500 && $4<600 {count[$3]++}
        END {for (path in count) printf "%s\t%d\n", path, count[path]}' \
    "$csv_file" | sort -k2,2nr -k1,1 | head -n 2

avg=$(awk -F, 'NR>1 {sum += $5; n++}
        END {if(n>0) printf "%.2f", sum/n; else print "0.00"}' \
    "$csv_file")
echo "Average latency: $avg"
EOF

chmod +x analyze.sh
./analyze.sh access.csv
./analyze.sh nothing.csv
echo $?
```

A terminal window on a Kali Linux system. The prompt is (kali@kali)-[~/Desktop/新建文件夹/q02]. The user runs ./analyze.sh nothing.csv, which results in an error: Error: File 'nothing.csv' not found. Then, the user runs ./analyze.sh access.csv. The script outputs: Top 2 paths with most 5xx errors: carol 2 bob 2, and Average latency: 193.75. The terminal background has a dark, abstract pattern.

```
(kali@kali)-[~/Desktop/新建文件夹/q02]
$ ./analyze.sh nothing.csv
Error: File 'nothing.csv' not found

(kali@kali)-[~/Desktop/新建文件夹/q02]
$ ./analyze.sh access.csv
Top 2 paths with most 5xx errors:
carol 2
bob 2
Average latency: 193.75

(kali@kali)-[~/Desktop/新建文件夹/q02]
$
```

图 2 实验二运行结果

结果说明

脚本通过管道完成 5xx 路径计数和排序，并输出前 2 项；全部数据行的平均延迟为 193.75 ms。文件不存在时，错误信息写入 stderr，并返回非零退出码。

实验三 制造并解决一次合并冲突 (Version Control and Git)

执行命令

```
mkdir q03 && cd q03
git init -b main
printf 'mode=normal\n' > config.txt
git add config.txt
git commit -m 'add config.txt'
BASE=$(git rev-parse HEAD)

git switch -c feature-a
printf 'mode=safe\n' > config.txt
git add config.txt
git commit -m 'modify config.txt'

git switch -c feature-b "$BASE"
printf 'mode=fast\n' > config.txt
git add config.txt
git commit -m 'modify config.txt again'

git switch main
git merge feature-a
git merge feature-b
printf 'mode=safe\nnote=reviewed\n' > config.txt
git add config.txt
git commit -m 'merge feature-b'

git status
git log --all --graph --decorate --oneline
```

```
kali@kali: ~/Desktop/q03
$ git add config.txt
$ git commit -m "merge feature-b"
[main a58ead4] merge feature-b
$ git status
位于分支 main
无文件要提交，干净的工作区
$ git log
commit a58ead45146c4969c7788ac24b0da6a9a86c5f85 (HEAD -> main)
Merge: 4e2dfb9 d97957c
Author: FinaleQuill <166838202@qq.com>
Date: Mon Aug 24 03:51:06 2026 -0400

    merge feature-b

commit d97957ca14cde135cbe1b12c99567fa1a0ff5b (feature-b)
Author: FinaleQuill <166838202@qq.com>
Date: Mon Aug 24 03:48:18 2026 -0400

    modify config.txt again

commit 4e2dfb92129818b97008a93d385638157f3030d5 (feature-a)
Author: FinaleQuill <166838202@qq.com>
Date: Mon Aug 24 03:46:57 2026 -0400

    modify config.txt

commit f83fb2d370b886f5ebb6504f9622802d730ca84
Author: FinaleQuill <166838202@qq.com>
Date: Mon Aug 24 03:45:33 2026 -0400

    add config.txt
$ git log --all --graph --decorate --oneline
* a58ead4 (HEAD -> main) merge feature-b
|
| * d97957c (feature-b) modify config.txt again
| * 4e2dfb9 (feature-a) modify config.txt
|/
* f83fb2d add config.txt
$
```

图 3 实验三运行结果

结果说明

两个特性分支对同一配置项写入不同值并产生合并冲突。冲突解决后，config.txt 保留 mode=safe 和 note=reviewed，工作区状态干净，提交图完整显示分支与合并过程。

实验四 修复并构建一页技术说明 (LaTeX 文档编辑)

执行命令

```
mkdir q04 && cd q04
cat > report.tex <<'EOF'
\documentclass[11pt]{article}
\usepackage[a4paper,margin=2.4cm]{geometry}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage{xcolor}
\usepackage[hidelinks]{hyperref}

\definecolor{accent}{HTML}{1F5A94}
\setlength{\parindent}{0pt}
\setlength{\parskip}{0.65em}

\begin{document}
\title{\textcolor{accent}{Tool Report}}
\author{Zhu Yuanqing -- 25020007193}
\date{August 24, 2026}
\maketitle

\section{Result}
For a data set containing  $n$  latency measurements  $t_i$ , the arithmetic
mean is defined by Equation~\eqref{eq:mean-latency}.
\begin{equation}
\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i
\label{eq:mean-latency}
\end{equation}
The build check in Table~\ref{tab:build-check} records the two essential
parts of this one-page technical note.

\begin{table}[htbp]
\centering
\caption{Build check}
\label{tab:build-check}
\begin{tabular}{cc}
\toprule
Item & Status \\
\midrule
Formula reference & Passed \\
\bottomrule
\end{tabular}
\end{table}
\end{document}
EOF
```

tectonic report.tex

1 Result

For a data set containing n latency measurements t_i , the arithmetic mean is defined by Equation (1).

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i$$

(1)

The compact build check in Table 1 records the two essential parts of this one-page technical note.

Table 1: Build check	
Item	Status
Formula reference	Passed

The numbered formula and the captioned table are both referenced in the body text. The document is therefore ready for a command-line build.

图 4 实验四构建结果

结果说明

命令行构建生成 report.pdf。编号公式和 2 行 2 列表格均设置了 label，正文引用显示实际编号，PDF 可以正常打开。

课后练习

Shell 命令实例（选做 10 题）

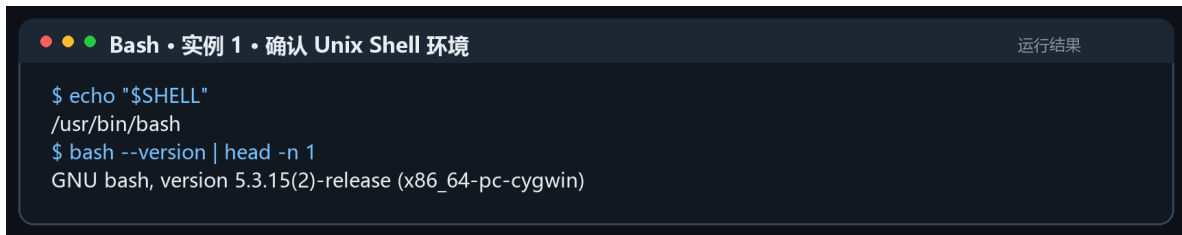
表 2 课后练习选题

编号	实例内容	关键命令
1	确认 Unix Shell 环境	echo、bash
2	读取 ls -l 权限字段	ls -l
3	测试 Shell glob 模式	*, ?、花括号展开
4	比较三种引号	单引号、双引号、ANSI-C 引号
5	重定向标准输出与错误	>、2>、2>&1
6	退出状态与条件执行	\$?、&&、
7	验证 cd 是内建命令	type、子 Shell
8	用脚本检查文件是否存在	test -f
10	使用 set -x 跟踪命令	set -x
11	生成带日期的备份文件名	\$(date ...)

实例 1 确认 Unix Shell 环境

执行命令

```
echo "$SHELL"
bash --version | head -n 1
```



The screenshot shows a terminal window titled "Bash · 实例 1 · 确认 Unix Shell 环境". The terminal displays the following commands and their outputs:

```
$ echo "$SHELL"
/usr/bin/bash
$ bash --version | head -n 1
GNU bash, version 5.3.15(2)-release (x86_64-pc-cygwin)
```

图 5 实例 1 运行结果

结果说明

Shell 路径为 `/usr/bin/bash`，版本信息由 Bash 正常输出，当前环境满足 Unix 风格 Shell 的使用要求。

实例 2 读取 ls -l 权限字段

执行命令

```
ls -l /  
ls -ld / /tmp /usr/bin/bash
```



The screenshot shows a terminal window titled "Bash · 实例 2 · 读取 ls -l 权限字段" with a "运行结果" (Run Result) button. The terminal output is as follows:

```
$ ls -l /  
total 2440  
-rw-r--r-- 1 16683 197610 18765 Aug 21 00:03 LICENSE.txt  
-rw-r--r-- 1 16683 197610 3738 Aug 21 00:03 README.portable  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:15 bin  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:15 cmd  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:14 dev  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:15 etc  
-rwxr-xr-x 1 16683 197610 135016 Aug 20 23:47 git-bash.exe  
-rwxr-xr-x 1 16683 197610 134488 Aug 20 23:47 git-cmd.exe  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:14 mingw64  
dr-xr-xr-x 8 16683 197610 0 Aug 24 20:19 proc  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:19 tmp  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:14 usr  
$ ls -ld / /tmp /usr/bin/bash  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:15 /  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:19 /tmp  
-rwxr-xr-x 1 16683 197610 2449886 Aug 20 22:55 /usr/bin/bash
```

图 6 实例 2 运行结果

结果说明

-l 使用长格式显示文件类型、权限、链接数、所有者、组、大小和时间等信息。每行前 10 个字符中，第 1 个字符表示文件类型；第 2-4、5-7、8-10 个字符分别表示所有者、用户组和其他用户的读、写、执行权限。

实例 3 测试 Shell glob 模式

执行命令

```
mkdir -p glob-test && cd glob-test
touch a.txt b.txt c.txt file1.txt file2.txt file10.txt fileA.txt note.md
ls *.txt
ls file?.txt
ls {a,b,c}.txt
```



```
Bash · 实例 3 · 测试 Shell glob 模式 运行结果

$ ls *.txt
a.txt
b.txt
c.txt
file1.txt
file10.txt
file2.txt
fileA.txt
$ ls file?.txt
file1.txt
file2.txt
fileA.txt
$ ls {a,b,c}.txt
a.txt
b.txt
c.txt
```

图 7 实例 3 运行结果

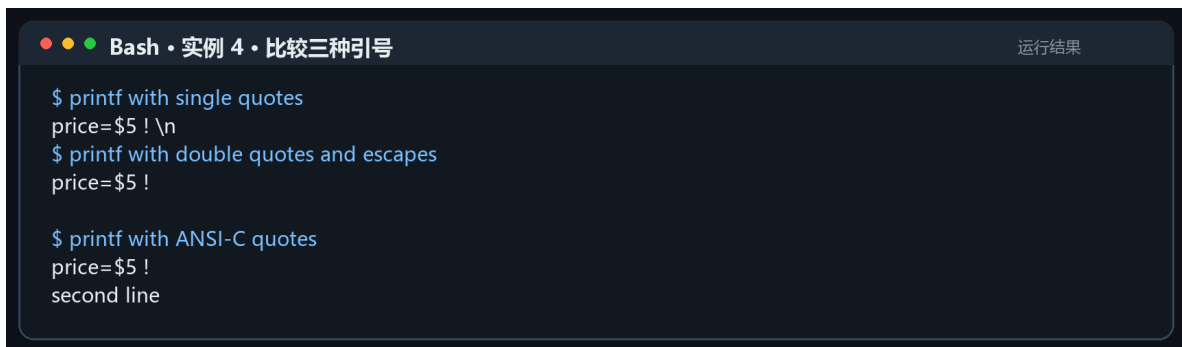
结果说明

* 匹配任意长度的字符串，? 恰好匹配一个字符；{a,b,c}.txt 先展开为三个明确的文件名，因此三种模式得到不同的文件集合。

实例 4 比较三种引号

执行命令

```
printf '%s\n' 'price=$5 ! \n'  
printf "%s\n" "price=\$5 ! \n"  
printf '%s' '$'price=$5 !\nsecond line\n'
```



The screenshot shows a terminal window titled "Bash · 实例 4 · 比较三种引号" with a "运行结果" (Execution Result) button. It displays the output of three printf commands: 1. Single quotes: prints "price=\$5 ! \n" on one line. 2. Double quotes and escapes: prints "price=\$5 !" on one line. 3. ANSI-C quotes: prints "price=\$5 !" on one line and "second line" on the next line.

```
Bash · 实例 4 · 比较三种引号 运行结果  
$ printf with single quotes  
price=$5 ! \n  
$ printf with double quotes and escapes  
price=$5 !  
  
$ printf with ANSI-C quotes  
price=$5 !  
second line
```

图 8 实例 4 运行结果

结果说明

单引号保留全部字符的字面值；双引号允许变量等展开，因此需要反斜杠保留美元符号；ANSI-C 引号会解释 `\n`，从而输出实际换行。

实例 5 重定向标准输出与错误

执行命令

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
head -n 8 stdout.txt
cat stderr.txt
ls /nonexistent /tmp > both.txt 2>&1
head -n 10 both.txt
```

```
Bash · 实例 5 · 重定向标准输出与错误 运行结果

$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt
$ head -n 8 stdout.txt
/tmp:
00
0021c146-98db-4f9e-9c49-651c12b83f17.tmp
01
0152d80c-bc47-4921-9843-3318959c5ece.tmp
02
02kwl3wq.qq1
03
$ cat stderr.txt
ls: cannot access '/nonexistent': No such file or directory
$ ls /nonexistent /tmp > both.txt 2>&1
$ head -n 10 both.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
00
0021c146-98db-4f9e-9c49-651c12b83f17.tmp
01
0152d80c-bc47-4921-9843-3318959c5ece.tmp
02
02kwl3wq.qq1
03
04
```

图 9 实例 5 运行结果

结果说明

> 将 stdout 写入 stdout.txt, 2> 将 stderr 写入 stderr.txt; 2>&1 使 stderr 指向与 stdout 相同的目标, 两类信息均进入 both.txt。

实例 6 退出状态与条件执行

执行命令

```
false; echo $?  
true && echo success  
false || echo fallback  
target="mydir_25020007193_$$"  
test -d "$target" || mkdir "$target"  
ls -ld "$target"
```



```
Bash · 实例 6 · 退出状态与条件执行 运行结果  
$ false; echo $?  
1  
$ true && echo success  
success  
$ false || echo fallback  
fallback  
$ test -d "$target" || mkdir "$target"  
$ ls -ld "$target"  
drwxr-xr-x 1 16683 197610 0 Aug 24 20:19 mydir_25020007193_2001
```

图 10 实例 6 运行结果

结果说明

`false` 的退出状态为 1；`&&` 仅在前一条命令成功时继续，`||` 仅在失败时继续。目录不存在时，`test -d` 返回非零状态并触发 `mkdir`。

实例 7 验证 cd 是 Shell 内建命令

执行命令

```
mkdir -p ex07/subdir && cd ex07
type cd
pwd
(cd subdir; pwd)
pwd
```



The screenshot shows a terminal window titled "Bash · 实例 7 · 验证 cd 是 Shell 内建命令". The terminal output is as follows:

```
$ type cd
cd is a shell builtin
$ pwd
/c/Users/16683/Desktop/系统开发工具基础/after_class_practice/runs/ex07
$ (cd subdir; pwd)
/c/Users/16683/Desktop/系统开发工具基础/after_class_practice/runs/ex07/subdir
$ pwd
/c/Users/16683/Desktop/系统开发工具基础/after_class_practice/runs/ex07
```

图 11 实例 7 运行结果

结果说明

type cd 显示 cd 是 Shell 内建命令。子 Shell 能改变自己的工作目录，但结束后父 Shell 的目录保持不变；因此 cd 必须在当前 Shell 中直接修改目录状态。

实例 8 用脚本检查文件是否存在

执行命令

```
cat > check.sh <<'EOF'
#!/usr/bin/env bash
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <file>" >&2
    exit 2
fi
if [ -f "$1" ]; then
    echo "File exists: $1"
else
    echo "File not found: $1" >&2
    exit 1
fi
EOF

printf 'sample\n' > existing.txt
bash check.sh existing.txt
bash check.sh missing.txt
echo $?
```



```
Bash · 实例 8 · 用脚本检查文件是否存在 运行结果

$ bash check.sh existing.txt
File exists: existing.txt
$ bash check.sh missing.txt
File not found: missing.txt
$ echo $?
1
```

图 12 实例 8 运行结果

结果说明

脚本用 [-f "\$1"] 判断参数是否指向普通文件；存在时输出成功提示，不存在时向 stderr 输出提示并返回退出状态 1。

实例 10 使用 set -x 跟踪命令

执行命令

```
cat > trace.sh <<'EOF'
#!/usr/bin/env bash
set -x
name="trace-demo"
echo "hello $name"
test -f sample.txt || touch sample.txt
wc -c sample.txt
EOF

rm -f sample.txt
bash trace.sh
```



```
Bash · 实例 10 · 使用 set -x 跟踪命令 运行结果

$ bash trace.sh
+ name=trace-demo
+ echo 'hello trace-demo'
hello trace-demo
+ test -f sample.txt
+ touch sample.txt
+ wc -c sample.txt
0 sample.txt
```

图 13 实例 10 运行结果

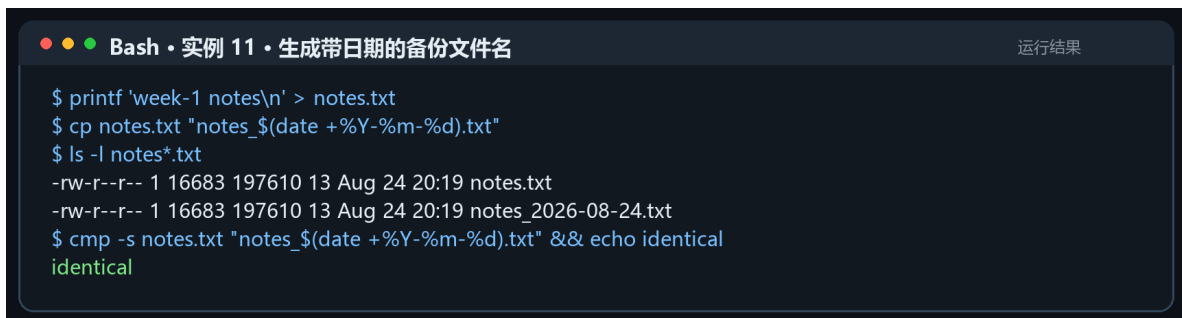
结果说明

set -x 在命令执行前输出展开后的命令，并以 + 标记跟踪行；变量展开、条件判断和创建文件的执行顺序均可直接观察。

实例 11 生成带日期的备份文件名

执行命令

```
printf 'week-1 notes\n' > notes.txt
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ls -l notes*.txt
cmp -s notes.txt "notes_$(date +%Y-%m-%d).txt" && echo identical
```



The screenshot shows a terminal window titled "Bash • 实例 11 • 生成带日期的备份文件名" with a "运行结果" (Run Result) button. The terminal displays the following commands and their output:

```
$ printf 'week-1 notes\n' > notes.txt
$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
$ ls -l notes*.txt
-rw-r--r-- 1 16683 197610 13 Aug 24 20:19 notes.txt
-rw-r--r-- 1 16683 197610 13 Aug 24 20:19 notes_2026-08-24.txt
$ cmp -s notes.txt "notes_$(date +%Y-%m-%d).txt" && echo identical
identical
```

图 14 实例 11 运行结果

结果说明

命令替换 `$(date +%Y-%m-%d)` 生成当天日期并嵌入备份文件名；`cmp -s` 验证源文件与备份内容完全一致。